2. Keylime 개요 및 아키텍처

2.1 Keylime 개요

2.1.1 Keylime의 목적과 핵심 기능

Keylime은 TPM(Trusted Platform Module) 기술을 활용한 오픈소스 확장 가능한 신뢰 시스템입니다. 하드웨어 기반 암호화 신뢰를 제공하여 원격 머신의 무결성을 검증하고 암호화된 페이로드를 안전하게 배포하는 엔드투엔드 솔루션을 제공합니다.

핵심 기능

- **하드웨어 기반 신뢰 부트스트래핑**: TPM을 활용한 원격 머신의 암호화 신뢰 구축
- **암호화된 페이로드 프로비저닝**: 신뢰가 확립된 후 암호화된 데이터 안전 배포
- **런타임 시스템 무결성 모니터링**: 지속적인 시스템 상태 검증
- **유연한 원격 증명 프레임워크**: 사용자 정의 PCR(Platform Configuration Register) 증명
- **사용자 정의 액션 트리거**: 측정값 실패 시 자동화된 대응 메커니즘

적용 시나리오

- **하이브리드 클라우드 환경**: 완전한 제어권이 없는 원격 머신의 증명
- **엣지/IoT 디바이스**: 물리적으로 안전하지 않은 환경의 원격 디바이스 관리
- **보안이 중요한 워크로드**: 하드웨어 기반 신뢰가 필요한 애플리케이션

2.1.2 Keylime의 미션

Keylime의 미션은 TPM 기술을 개발자와 사용자가 쉽게 접근할 수 있도록 하는 것입니다. TPM의 저수준 동작에 대한 깊은 이해 없이도 하드웨어 기반 신뢰를 활용할 수 있는 환경을 제공합니다.

2.2 시스템 아키텍처

2.2.1 전체 시스템 아키텍처 다이어그램

```
┌─────────────────────────────────────────────────────────────────────────────────────────┐
| Keylime 시스템 아키텍처 |
└─────────────────────────────────────────────────────────────────────────────────────────┘

┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
| Tenant | | Registrar | | Verifier |
| (관리 도구) | | (등록 서버) | | (검증 서버) |
| | | | | |
| • CLI 인터페이스 |◄────────┤ • TPM 공개키 |◄────────┤ • 지속적 검증 |
| • RESTful API | | 관리 | | • 무결성 모니터링|
| • 정책 관리 | | • 에이전트 등록 | | • 상태 추적 |
| • 페이로드 암호화| | • EK/AIK 검증 | | • 실패 처리 |
└──────────────────┘ └──────────────────┘ └──────────────────┘
| | |
| | |
└──────────────────────────┼──────────────────────────┘
|
┌──────────────────┐
| Agent |
```

```
| (검증 대상) |
| |
| • TPM 통신 |
| • 측정값 수집 |
| • 증명 생성 |
| • 페이로드 복호화|
└──────────────────┘
|
┌──────────────────┐
| TPM |
| (하드웨어 보안) |
| |
| • PCR 관리 |
| • 키 생성/보관 |
| • 암호화 연산 |
| • 증명 서명 |
└──────────────────┘


┌──────────────────────────────────────────────────────────────────────────────┐
| 통신 프로토콜 |
| |
| ◀── HTTPS/TLS ──▶ ◀── mTLS ──▶ ◀── HTTPS/TLS ──▶ |
| (RESTful API) (상호 인증) (RESTful API) |
```

└──┘

2.2.2 컴포넌트 간 데이터 흐름

```
등록 단계:

Agent → Registrar: EK, AIK 공개키 등록

Registrar → Agent: 등록 확인


프로비저닝 단계:

Tenant → Verifier: 에이전트 추가 요청 (정책, 페이로드)

Verifier → Registrar: 에이전트 공개키 조회

Verifier → Agent: 신뢰 설정 및 페이로드 전송


검증 단계 (지속적):

Verifier → Agent: Quote 요청 (PCR 측정값)

Agent → Verifier: TPM Quote 응답

Verifier: 정책 대비 검증 수행
```

2.3 주요 컴포넌트 역할 정의

2.3.1 Verifier (검증자)

파일 위치: `keylime/cloud_verifier_tornado.py` , `keylime/cloud_verifier_common.py`

핵심 역할

- **지속적 무결성 검증**: 에이전트가 실행되는 머신의 무결성 상태를 지속적으로 검증
- **암호화 키 부트스트래핑**: 에이전트와 안전한 암호화 키 교환 지원
- **정책 기반 검증**: TPM 정책 및 IMA(Integrity Measurement Architecture) 정책 검증
- **실패 처리**: 측정값 실패 시 자동화된 대응 및 알림

주요 기능

```
# keylime/cloud_verifier_tornado.py의 주요 클래스
class AgentsHandler(BaseHandler):
def  get(self): # 에이전트 상태 조회
def  post(self): # 에이전트 등록
def  delete(self): # 에이전트 삭제
```

기술적 특징

- **Tornado 웹 프레임워크**: 비동기 처리를 통한 고성능 구현
- **상호 TLS 인증**: 보안 통신을 위한 mTLS 지원
- **데이터베이스 연동**: SQLAlchemy ORM을 통한 에이전트 상태 관리
- **세션 관리**: `SessionManager` 를 통한 데이터베이스 세션 관리

2.3.2 Registrar (등록자)

파일 위치: `keylime/registrar_client.py`

핵심 역할

- **TPM 공개키 데이터베이스**: 모든 에이전트의 TPM 공개키 저장 및 관리
- **TPM 벤더 공개키 호스팅**: TPM 제조사의 공개키 저장

- **에이전트 등록 관리**: 새로운 에이전트의 등록 및 인증 처리
- **키 검증 인터페이스**: Quote 검증을 위한 공개키 제공

주요 데이터 구조

```
# keylime/registrar_client.py의 RegistrarData TypedDict
class RegistrarData(TypedDict):
ip: Optional[str] # 에이전트 IP 주소
port: Optional[str] # 에이전트 포트
regcount: int # 등록 카운터
mtls_cert: Optional[str] # mTLS 인증서
aik_tpm: str # AIK (Attestation Identity Key)
ek_tpm: str # EK (Endorsement Key)
ekcert: Optional[str] # EK 인증서
provider_keys: NotRequired[Dict[str, str]] # 제공자 키
```

핵심 함수

```
def  getData(registrar_ip: str, registrar_port: str,
agent_id: str, tls_context: Optional[ssl.SSLContext]) -> Optional[RegistrarData]
"""에이전트 데이터를 Registrar에서 조회"""
```

2.3.3 Agent (에이전트)

파일 위치: `keylime/agentstates.py` (상태 관리)

핵심 역할

- **TPM 통신**: 로컬 TPM과의 직접적인 통신 및 제어
- **측정값 수집**: 시스템 상태 및 무결성 측정값 수집
- **증명 생성**: TPM을 사용한 디지털 증명(Quote) 생성
- **페이로드 관리**: 암호화된 페이로드 수신 및 복호화

상태 관리 구조

```
# keylime/agentstates.py의 TPMState 클래스
class TPMState:
pcrs: Dict[int, Optional[bytes]] # PCR 값들
hash_alg: Dict[int, Hash] # PCR별 해시 알고리즘

# TPM 클록 정보
@dataclass
class TPMClockInfo:
clock: int # TPM 클록
resetcount: int # 리셋 카운터
restartcount: int # 재시작 카운터
safe: int # 안전 플래그
```

에이전트 증명 상태

```
class AgentAttestState:
```

```
agent_id: str # 에이전트 식별자

next_ima_ml_entry: int # 다음 IMA 측정 로그 엔트리

boottime: int # 부팅 시간

tpm_clockinfo: TPMClockInfo # TPM 클록 정보

tmp_state: TPMState # TPM 상태

ima_pcrs: Set[int] # IMA PCR 집합

ima_keyring: ImaKeyrings # IMA 키링
```

2.3.4 Tenant (테넌트)

파일 위치: `keylime/tenant.py`

핵심 역할

- **명령줄 관리 도구**: Keylime 시스템의 주요 관리 인터페이스
- **에이전트 프로비저닝**: 새로운 에이전트 등록 및 설정
- **정책 관리**: TPM 정책 및 IMA 정책 설정
- **페이로드 암호화**: 안전한 데이터 전송을 위한 암호화

주요 클래스 구조

```
class Tenant:

# 네트워크 설정

verifier_ip: Optional[str]

verifier_port: Optional[str]

agent_ip: Optional[str]

agent_port: Optional[str]
```

```
registrar_ip: Optional[str]

registrar_port: Optional[str]

# 암호화 키

K: bytes = b""  # 암호화 키

V: bytes = b""  # 검증 키

U: bytes = b""  # UUID 키

# 정책 설정

tpm_policy = None # TPM 정책

runtime_policy: str = ""  # 런타임 정책

ima_sign_verification_keys: Optional[str] = ""  # IMA 서명 검증 키
```

2.4 TPM 2.0 지원 및 구현

2.4.1 TPM 2.0 지원

Keylime은 TPM 버전 2.0을 지원하며, 다음과 같은 특징을 가집니다:

TPM 2.0 핵심 기능 지원

- **PCR (Platform Configuration Register)**: 24개 PCR 지원 (0-23번)
- **EK (Endorsement Key)**: TPM 제조사가 생성한 고유 키
- **AIK (Attestation Identity Key)**: 증명용 신원 키
- **Quote 생성**: TPM을 사용한 디지털 서명된 증명

TPM 메인 모듈 구현

```
# keylime/tpm/tpm_main.py의 Tpm 클래스

class Tpm:

@staticmethod

def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tmp: bytes) -> Optional[T

"""EK를 사용하여 AIK 암호화"""

aik_name = tpm2_objects.get_tpm2b_public_name(aik_tpm)

challenge_str = tpm_util.random_password(32)

challenge = challenge_str.encode()

# MakeCredential 프로토콜 사용

credentialblob = tpm_util.makecredential(ek_tpm, challenge, bytes.fromhex(aik_na

keyblob = base64.b64encode(credentialblob)

return (keyblob, key)
```

2.4.2 하드웨어 TPM vs 소프트웨어 TPM

하드웨어 TPM (권장)

특징:

- **진정한 하드웨어 신뢰 루트**: 물리적 보안 칩에서 실행
- **변조 방지**: 물리적 공격에 대한 저항성
- **프로덕션 환경 필수**: 실제 보안이 필요한 환경에서 사용

보안 이점:

- 키가 하드웨어에서 생성되고 보호됨
- PCR 값이 하드웨어에서 안전하게 관리됨

- 물리적 변조 시도 시 키 삭제

소프트웨어 TPM (개발/테스트용)

특징:

- **개발 및 테스트 목적**: 실제 TPM 하드웨어가 없는 환경에서 사용
- **에뮬레이션**: TPM 기능을 소프트웨어로 구현
- **보안 제한**: 하드웨어 신뢰 루트 제공하지 않음

중요한 보안 경고:

```
⚠️ 경고: 프로덕션 환경에서 소프트웨어 TPM 에뮬레이터 사용 금지!
소프트웨어 TPM은 하드웨어 신뢰 루트를 제공하지 않으며,
Keylime 사용의 보안 이점을 크게 감소시킵니다.
실제 비밀과 신뢰가 필요한 경우 반드시 하드웨어 TPM을 사용해야 합니다.
```

TPM 지원 확인

```
# keylime/config.py에서 TPM 에뮬레이터 설정
DISABLE_EK_CERT_CHECK_EMULATOR = False # 에뮬레이터 EK 인증서 검사 비활성화
TEST_MODE = environ_bool("KEYLIME_TEST", False) # 테스트 모드 설정
```

2.4.3 TPM 2.0 암호화 알고리즘 지원

지원되는 해시 알고리즘

- **SHA-1**: 레거시 지원
- **SHA-256**: 기본 권장 알고리즘

- **SHA-384**: 고급 보안 요구사항
- **SHA-512**: 최고 보안 수준

지원되는 암호화 알고리즘

- **RSA**: 2048비트, 3072비트, 4096비트
- **ECC (Elliptic Curve Cryptography)**: P-256, P-384, P-521

서명 알고리즘

- **RSASSA-PKCS1-v1_5**: RSA 서명
- **RSASSA-PSS**: RSA PSS 서명
- **ECDSA**: 타원곡선 디지털 서명

2.5 데이터베이스 스키마

2.5.1 주요 데이터베이스 테이블

VerfierMain 테이블

```
# keylime/db/verifier_db.py

class VerfierMain(Base):

__tablename__ = "verifiermain"

agent_id = Column(String(80), primary_key=True) # 에이전트 식별자

operational_state = Column(Integer) # 운영 상태

tpm_policy = Column(JSONPickleType(pickler=JSONPickler)) # TPM 정책

ima_policy = relationship("VerifierAllowlist") # IMA 정책 관계

mb_policy = relationship("VerifierMbpolicy") # 측정 부트 정책 관계
```

```
boottime = Column(Integer) # 부팅 시간

ima_pcrs = Column(JSONPickleType(pickler=JSONPickler)) # IMA PCR 목록

tpm_clockinfo = Column(JSONPickleType(pickler=JSONPickler)) # TPM 클록 정보
```

이 문서는 Keylime TPM 검증 시스템의 전체적인 개요와 아키텍처를 제공하며, 각 컴포넌트의 역할과 TPM 2.0 지원에 대한 상세한 정보를 포함합니다.